

Cidades ESG Inteligentes

CI/CD, containerização e orquestração de uma API Java Spring Boot

Integrante

Sidney — RM: _____

Repositório

https://github.com/_____/cidades-esg-inteligentes

Stack

Java 21 • Spring Boot 3.3 • PostgreSQL 16

Docker (multi-stage) • Docker Compose v2

GitHub Actions • GitHub Container Registry • Environments

1. Visão geral da solução

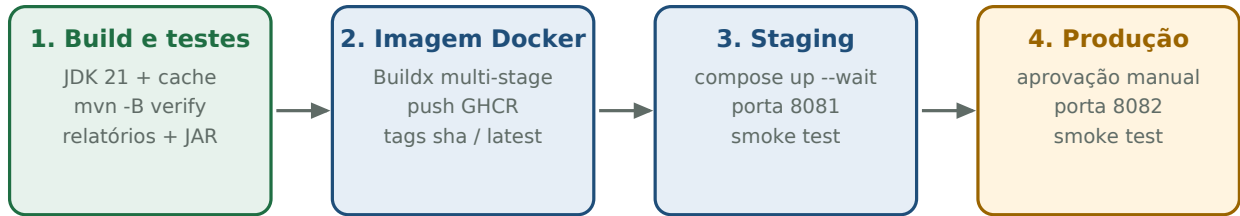
A aplicação é uma API REST que monitora o consumo de energia de edifícios públicos. Cada edifício tem uma **meta diária de kWh**; as leituras dos medidores são somadas por dia e, quando a soma ultrapassa a meta, a API abre um alerta. O endpoint `/api/indicadores` consolida consumo total, emissão estimada de CO₂ (fator médio do SIN) e alertas abertos/resolvidos — o pilar Ambiental do ESG aplicado à cidade.

Sobre essa API foi montado o ciclo DevOps pedido no desafio: testes automatizados, imagem Docker, orquestração com Docker Compose e pipeline no GitHub Actions que entrega a mesma imagem em staging e produção.

Camada	Componentes
Domínio	Edificio, MedicaoConsumo, AlertaConsumo (JPA)
Regra de negócio	MonitoramentoEnergeticoService: soma diária, abertura/atualização de alerta, indicadores ESG
API	EdificioController, AlertaController, InfoController (/api/info informa o ambiente)
Observabilidade	Spring Actuator /actuator/health, usado pelo Docker HEALTHCHECK e pelo smoke test
Testes	4 unitários (JUnit 5 + Mockito) e 4 de integração (SpringBootTest + MockMvc + H2)

2. Pipeline CI/CD

Ferramenta: GitHub Actions, workflow `.github/workflows/ci-cd.yml`. **Gatilhos:** push na main (ciclo completo), pull request (só build e testes) e execução manual.



push na main dispara as 4 etapas • pull request executa só a etapa 1

Etapa	Lógica
1. Build e testes	Configura JDK 21 com cache do Maven e executa <code>mvn -B verify</code> : compila, roda os 8 testes e gera o JAR. Se um teste falhar, nada segue adiante. Relatórios Surefire e JAR viram artefatos; o resumo aparece na página do run.
2. Imagem Docker	Buildx gera a imagem multi-stage com cache do GitHub e publica no GHCR com duas tags: o SHA curto do commit (rastreador) e <code>latest</code> . Não roda em pull request.
3. Deploy staging	Environment staging . Faz pull da imagem exata do commit e executa <code>scripts/deploy.sh staging</code> , que sobe API + PostgreSQL com <code>deploy/.env.staging</code> e espera os health checks (<code>--wait</code>). O smoke test confere health, ambiente = staging e o fluxo de alerta. Logs e status dos containers são publicados como evidência.
4. Deploy produção	Só roda com staging verde. Environment production com revisor obrigatório: o run pausa até a aprovação. Mesmo processo com <code>deploy/.env.production</code> , porta 8082 e smoke test esperando ambiente = production.

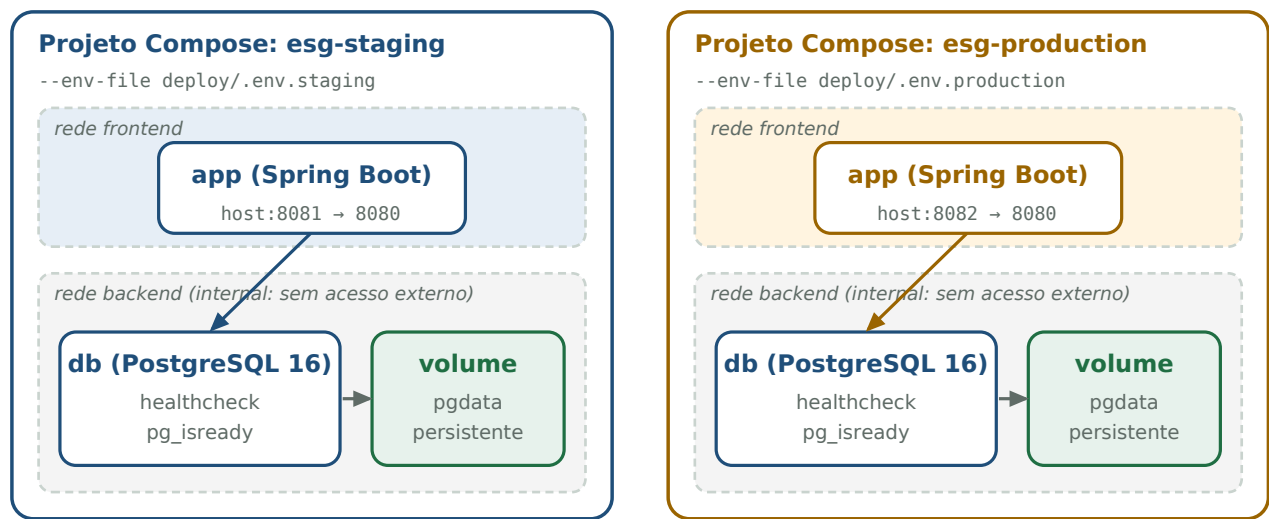
Decisões de projeto

- **Build once, deploy many:** staging e produção recebem a mesma imagem (mesmo SHA). O que muda entre eles são só as variáveis do ambiente.
- **Portão de qualidade em dois níveis:** testes antes da imagem e smoke test após cada deploy. Produção depende do smoke test de staging.
- **Segredos fora do código:** `DB_PASSWORD` é secret de cada environment do GitHub; os arquivos versionados não têm senha.
- **Runner configurável:** a variável `DEPLOY_RUNNER` permite apontar os deploys para um runner self-hosted, onde os ambientes continuam no ar depois do job.

3. Docker

Arquitetura dos ambientes

O mesmo `docker-compose.yml` cria ambientes isolados: cada um é um projeto Compose próprio (`-p esg-staging / -p esg-production`), com suas redes, seu volume de dados e sua porta.



Recurso	Como foi usado
Volumes	pgdata guarda os dados do PostgreSQL e sobrevive a recriações do container
Redes	backend é internal (banco sem rota para fora); frontend apenas publica a porta da API
Variáveis de ambiente	<code>.env</code> (local), <code>deploy/.env.staging</code> , <code>deploy/.env.production</code> ; Spring lê <code>DB_URL</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>APP_ENV</code> , <code>JPA_DDL_AUTO</code> , <code>JAVA_OPTS</code>
Dependência e saúde	app só inicia com o db healthy; a API tem HEALTHCHECK próprio

Dockerfile (imagem criada)

```
# =====
# Cidades ESG Inteligentes - imagem multi-stage
# Estagio 1: build com Maven | Estagio 2: runtime somente JRE
# =====

# ----- Estagio 1: build -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /workspace

# Copia apenas o pom primeiro para aproveitar o cache de dependencias
COPY pom.xml .
RUN mvn -B -q dependency:go-offline

COPY src ./src
# Os testes ja rodam no pipeline (job build-test); aqui so empacotamos
RUN mvn -B -q package -DskipTests

# ----- Estagio 2: runtime -----
FROM eclipse-temurin:21-jre-alpine

ARG APP_VERSION=dev
LABEL org.opencontainers.image.title="cidades-esg-inteligentes" \
      org.opencontainers.image.description="API de monitoramento energetico ESG" \
      org.opencontainers.image.version="${APP_VERSION}"

# Usuario sem privilegios (boa pratica de seguranca)
RUN addgroup -S esg && adduser -S esg -G esg
WORKDIR /app

COPY --from=build --chown=esg:esg /workspace/target/cidades-esg.jar app.jar

ENV APP_VERSION=${APP_VERSION} \
    TZ=America/Sao_Paulo \
    JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+UseG1GC"

USER esg
EXPOSE 8080

HEALTHCHECK --interval=15s --timeout=5s --start-period=40s --retries=5 \
  CMD wget -qO- http://localhost:8080/actuator/health | grep -q '"status":"UP"' || exit 1

ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

- **Multi-stage:** Maven + JDK só no estágio de build; a imagem final tem JRE Alpine e o JAR.
- **Cache de camadas:** pom.xml é copiado antes do código-fonte.
- **Segurança:** execução com usuário esg sem privilégios; .dockerignore exclui target, .git e .env.
- **Imagem publicada:** ghcr.io/<usuario>/cidades-esg-inteligentes:<sha> e :latest.

Comandos utilizados

```
# local
cp .env.example .env
docker compose up -d --build
docker compose ps
docker compose logs -f app
bash scripts/seed.sh http://localhost:8080

# staging e produção (mesmos comandos que o pipeline executa)
export DB_PASSWORD=*****
bash scripts/deploy.sh staging ghcr.io/<usuario>/cidades-esg-inteligentes:<sha>
bash scripts/smoke-test.sh http://localhost:8081 staging
bash scripts/deploy.sh production ghcr.io/<usuario>/cidades-esg-inteligentes:<sha>
bash scripts/smoke-test.sh http://localhost:8082 production

# imagem
docker build -t cidades-esg:local .
docker images cidades-esg
```

4. Evidências do pipeline

Prints do primeiro run completo no GitHub Actions (aba Actions do repositório).

INSERIR PRINT: pipeline completo

Aba Actions → run da main com os 4 jobs verdes

INSERIR PRINT: build e testes

Job 'Build e testes' → passo mvn verify com 'Tests run: 8, Failures: 0'

INSERIR PRINT: imagem publicada

Repositório → Packages → cidades-esg-inteligentes com as tags sha e latest

5. Ambientes staging e produção

INSERIR PRINT: deploy em staging

Job 'Deploy - staging' → passo smoke test terminando em 'SMOKE TEST OK (staging)'

INSERIR PRINT: aprovação de produção

Run pausado em 'Review deployments' → aprovação do environment production

INSERIR PRINT: deploy em produção

Job 'Deploy - producao' → 'SMOKE TEST OK (production)'

INSERIR PRINT: /api/info nos dois ambientes

curl localhost:8081/api/info e localhost:8082/api/info (ou arquivos *-info.json dos artefatos)

6. Desafios encontrados e soluções

Desafio	Solução adotada
Ambientes staging e produção na mesma infraestrutura sem conflito de containers, portas e dados	Um único compose parametrizado; cada ambiente roda como projeto Compose distinto (-p), com env-file, porta e volume próprios
Pipeline avançar com a aplicação ainda subindo	HEALTHCHECK no Dockerfile, depends_on com service_healthy e docker compose up --wait antes do smoke test
Banco de dados exposto desnecessariamente	Rede backend marcada como internal; só a API publica porta
Senha do banco no repositório	Secret DB_PASSWORD por environment do GitHub; compose falha se a variável não existir (\${DB_PASSWORD:?})
Runner do GitHub é descartado ao fim do job	Smoke test e coleta de logs no próprio job, publicados como artefatos; variável DEPLOY_RUNNER permite usar runner self-hosted persistente
GHCRC exige nome de imagem em minúsculas	Nome derivado de \${GITHUB_REPOSITORY,,} no próprio workflow
Mockito em modo estrito rejeita stubs não usados em parte dos testes	Stubs comuns declarados com lenient() e verificação com ArgumentCaptor
Soma de consumo é null quando não há leituras no dia	Tratamento explícito no serviço (null → zero) em vez de depender de coalesce do banco
Permissão de execução dos scripts perdida ao enviar o projeto em .zip	Pipeline chama os scripts com bash ./scripts/... em vez de depender do bit +x

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☒
Dockerfile funcional	☒
docker-compose.yml ou arquivos Kubernetes	☒
Pipeline com etapas de build, teste e deploy	☒
README.md com instruções e prints	☐
Documentação técnica com evidências (PDF ou PPT)	☐
Deploy realizado nos ambientes staging e produção	☐

Marcar os três últimos itens após inserir os prints do primeiro run do pipeline.